

Validation Report

Validation of UAV Path Following Algorithms Using Python

Paper Validated:

An Evaluation of UAV Path Following Algorithms

Submitted by:

Chandrakala Kotapati

USN: *ENG24AS0002*

Department: Aerospace Engineering

University: Dayananda Sagar University

Abstract

This report presents the validation of the IEEE paper **"An Evaluation of UAV Path Following Algorithms"** through independent implementation of five UAV guidance algorithms in Python. The implemented algorithms include Carrot Chasing, Nonlinear Guidance Law (NLGL), Pure Pursuit and LOS based path following (PLOS), Vector Field (VF), and Adaptive Linear Quadratic Regulator (LQR). The implementation was developed using Python with NumPy for numerical computation, Matplotlib for visualization, and Pandas for organizing the performance metrics. The reproduced simulations were performed under the same operating conditions and boundary conditions described in the reference paper. Performance was evaluated using Mean Cross-Track Error, Maximum Cross-Track Error, Root Mean Square Error (RMSE), Final Cross-Track Error, Settling Time, and comparative trajectory analysis. The reproduced trajectory plots demonstrate close qualitative agreement with the original publication for both straight-line and loiter path-following scenarios. Since the original numerical trajectory data are not publicly available, the validation is based on comparison of trajectory behavior, convergence characteristics, and quantitative performance metrics obtained from the implemented algorithms. The study demonstrates that the recreated algorithms successfully reproduce the published guidance behavior while providing a reproducible Python implementation for further research and educational use.

Introduction

Unmanned Aerial Vehicles (UAVs) have become increasingly important in modern aerospace engineering due to their ability to perform autonomous operations in various civilian and military applications. They are widely used for surveillance, environmental monitoring, disaster management, precision agriculture, infrastructure inspection, and search-and-rescue missions. To accomplish these tasks safely and efficiently, UAVs must be capable of following predefined flight paths accurately while maintaining stable and reliable flight. This makes the design of effective path-following guidance algorithms a critical aspect of autonomous UAV navigation.

Path following refers to the process of guiding a UAV along a specified geometric path while minimizing the deviation from the desired trajectory. Unlike trajectory tracking, path following focuses on maintaining the desired flight path without requiring the vehicle to reach specific points at predetermined times. An effective guidance algorithm should provide smooth convergence to the path, minimize cross-track error, ensure stable flight, and reduce unnecessary control effort. Therefore, selecting an appropriate guidance strategy is essential for improving the accuracy, efficiency, and robustness of UAV operations.

Several path-following algorithms have been developed to achieve these objectives, each with its own advantages and limitations. The algorithms considered in this validation study are **Carrot Chasing (CC)**, **Nonlinear Guidance Law (NLGL)**, **Pure pursuit and LOS based path following (PLOS)**, **Vector Field (VF)**, and **Adaptive Linear Quadratic Regulator (Adaptive LQR)**. These algorithms employ different mathematical approaches to generate steering commands and path corrections, resulting in varying levels of tracking accuracy, convergence speed, and control effort.

The reference paper, "**An Evaluation of UAV Path Following Algorithms**," presents a comparative analysis of these five guidance algorithms under straight-line and loiter (circular) path-following scenarios. The algorithms are evaluated using trajectory behavior, cross-track error, convergence characteristics, and control effort to identify their relative performance under identical simulation conditions. The study provides valuable insights into the strengths and limitations of each guidance method and serves as a useful benchmark for evaluating UAV guidance systems.

The objective of this work is to validate the results presented in the reference paper through an independent implementation of the five guidance algorithms in **Python**. The algorithms were developed using the mathematical models and simulation parameters described in the paper and evaluated under similar operating conditions. The reproduced simulations were analyzed using performance metrics such as **Mean Cross-Track Error (Mean XTE)**, **Maximum Cross-Track Error (Max XTE)**, **Root Mean Square Error (RMSE)**, **Final Cross-Track Error**, and **Settling Time**. The obtained trajectory plots and quantitative results were then compared

with those presented in the original paper to assess the accuracy and reproducibility of the implementation. This validation demonstrates that the implemented algorithms successfully reproduce the qualitative behavior reported in the reference study while providing a reproducible Python framework for further research and educational applications.

Validation summary

This report presents the validation of the research paper "**An Evaluation of UAV Path Following Algorithms**" through an independent implementation of the proposed guidance algorithms in **Python**. The algorithms implemented include **Carrot Chasing (CC)**, **Nonlinear Guidance Law (NLGL)**, **Pure pursuit and LOS based path following (PLOS)**, **Vector Field (VF)**, and **Adaptive Linear Quadratic Regulator (Adaptive LQR)**. The implementation was developed using **Python 3** with the **NumPy** library for numerical computations, **Matplotlib** for visualization of simulation results, and **Pandas** for organizing and comparing the performance metrics.

The mathematical models, simulation parameters, and boundary conditions described in the reference paper were followed as closely as possible to ensure a fair validation. Each guidance algorithm was evaluated for both **straight-line** and **loiter (circular)** path-following scenarios. The reproduced simulations were assessed using quantitative performance metrics, including **Mean Cross-Track Error (Mean XTE)**, **Maximum Cross-Track Error (Max XTE)**, **Root Mean Square Error (RMSE)**, **Final Cross-Track Error**, and **Settling Time**. A comparative benchmark was also created to evaluate the overall performance of the implemented algorithms under identical simulation conditions.

The reproduced trajectory plots exhibit **close qualitative agreement** with the figures presented in the reference paper. Similar convergence characteristics, path-following behavior, and relative performance trends were observed for all implemented guidance algorithms. Since the original paper does not provide the complete numerical simulation dataset, a direct point-by-point numerical comparison is not possible. Therefore, the validation is based on the agreement of trajectory behavior, qualitative analysis of the reproduced figures, and comparison of the calculated performance metrics. Overall, the implemented Python models successfully reproduce the behavior of the original guidance algorithms and provide a reliable framework for further analysis, experimentation, and educational use.

3. Governing Equations

3.1 UAV Kinematic Model

All guidance algorithms implemented in this study are based on a two-dimensional kinematic model of a fixed-wing UAV operating at a constant altitude and constant airspeed. The UAV is modeled as a point mass, and its state is represented by its position (x, y) and heading angle ψ . The vehicle motion is governed by

$$\begin{aligned} \dot{x} &= V_A \cos\psi \\ \dot{y} &= V_A \sin\psi \\ \dot{\psi} &= u \end{aligned}$$

where

- x, y = UAV position (m)
- V_A = Constant airspeed (m/s)
- ψ = Heading angle (rad)
- u = Heading rate command generated by the guidance algorithm.

In the numerical implementation, these continuous equations were solved using the forward Euler integration method.

3.2 Carrot Chasing Guidance Algorithm

The Carrot Chasing algorithm is a geometric path-following technique in which the UAV continuously tracks a **Virtual Target Point (VTP)** located ahead of the current projection point on the desired path. The target point is placed at a fixed look-ahead distance Δ , enabling smooth path convergence.

The projection distance along the desired path is

$$R = (x - x_i)\cos\theta + (y - y_i)\sin\theta$$

where

- (x_i, y_i) = Initial waypoint
- θ = Desired path angle

The Virtual Target Point coordinates are computed as

$$x_t = x_i + (R + \Delta)\cos\theta$$

$$y_t = y_i + (R + \Delta)\sin\theta$$

The desired heading angle is

$$\psi_d = \text{atan2}(y_t - y, x_t - x)$$

The heading error is

$$e_\psi = \psi_d - \psi$$

The commanded turn rate is then generated from the heading error while respecting the minimum turning radius constraint.

3.3 Nonlinear Guidance Law (NLGL)

The Nonlinear Guidance Law determines a virtual target by intersecting a guidance circle of radius L with the desired path. The heading correction depends on the angle between the UAV velocity vector and the look-ahead point.

The heading error is

$$\eta = \theta - \psi$$

The commanded turn rate is

$$u = \frac{2V_A \sin\eta}{L}$$

where

- L = Look-ahead distance
- V_A = UAV airspeed

- η = Heading error

The look-ahead distance is the primary tuning parameter of the algorithm. Smaller values provide faster convergence but may introduce oscillatory motion, whereas larger values produce smoother trajectories with slower convergence.

3.4 Pure Pursuit and LOS Based Path Following (PLOS)

The Pure Pursuit and LOS Based Path Following combines Pure Pursuit and Line-of-Sight (LOS) guidance to improve path-following performance. The controller simultaneously considers the heading error and cross-track error while generating the steering command.

The implemented steering law is

$$u = k_1(\theta - \psi) + k_2 d \cos(\theta - \theta_u)$$

where

- k_1 = Heading gain
- k_2 = Cross-track gain
- δ = Cross-track error
- θ = Desired path heading
- θ_u = UAV heading

The gain parameters determine the convergence rate and damping characteristics of the controller.

3.5 Vector Field Guidance

The Vector Field algorithm generates a desired heading by defining a continuous vector field around the reference path. Instead of directly pursuing a waypoint, the UAV follows the local vector direction, resulting in smooth and stable convergence.

The desired heading angle is

$$\chi_d = \theta_p - \chi_e \frac{2}{\pi} \tan^{-1}\left(\frac{k}{\tau} d\right)$$

where

- θ_p = Desired path heading
- d = Cross-track error
- k = Guidance gain
- τ = Path scaling factor
- χ_e = Heading correction coefficient

The arctangent function provides smooth convergence by gradually reducing the steering command as the vehicle approaches the desired path.

3.6 Adaptive Linear Quadratic Regulator (Adaptive LQR)

The Adaptive Linear Quadratic Regulator (Adaptive LQR) guidance algorithm formulates the path-following problem as an optimal control problem. Unlike conventional LQR, the controller adapts the state weighting matrix according to the current cross-track error, allowing the controller gains to vary during flight. This improves convergence while maintaining stable control effort. The implementation follows the adaptive LQR formulation presented in the reference paper.

The control law is given by

$$u = \left(\sqrt{q_{11}} d + \sqrt{\sqrt{q_{11}} + q_{22}} v_d \right)$$

Where

$$q_{11} = \left| \frac{d_b}{d_b - d} \right|$$

and

$$v_d = V_A \sin(\psi - \theta)$$

where

- d = Cross-track error
- d_b = Boundary distance around the reference path

- v_d = Cross-track velocity
- q_{22} = Weighting parameter of the state matrix
- V_A = Constant UAV airspeed
- ψ = UAV heading angle
- θ = Desired path heading

The state weighting matrix is

$$Q = \begin{bmatrix} q_{11} & 0 \\ 0 & q_{22} \end{bmatrix}$$

While

$$R = 1$$

The controller continuously updates q_{11} according to the vehicle position, allowing the control effort to increase when the UAV is far from the desired path and decrease as it converges. In the implemented simulations, different values of q_{22} were investigated to analyze their influence on convergence speed and tracking performance.

3.7 Boundary Conditions

To ensure a fair comparison among all guidance algorithms, identical simulation conditions were maintained throughout the study. The UAV was assumed to fly in a two-dimensional environment at a constant airspeed and constant altitude. Both straight-line and circular loiter path-following scenarios were simulated using the same initial conditions, simulation duration, and numerical integration timestep. The minimum turning radius constraint was enforced to represent realistic UAV maneuverability. Performance evaluation was carried out using Mean Cross-Track Error (Mean XTE), Maximum Cross-Track Error (Max XTE), Root Mean Square Error (RMSE), Final Cross-Track Error, and Settling Time, allowing a consistent comparison of the tracking performance of all implemented guidance algorithms.

4. Numerical Methods

The UAV path-following algorithms were independently implemented in **Python 3** based on the mathematical models and guidance laws presented in the reference paper. The implementation was carried out using **Google Colab**, which provided an interactive environment for numerical computation, simulation, visualization, and performance evaluation. The **NumPy** library was used for matrix operations and numerical calculations, **Matplotlib** was used to generate trajectory plots and comparative figures, and **Pandas** was used to organize the calculated performance metrics into tabular form.

A two-dimensional kinematic model of a fixed-wing UAV was adopted for all simulations. The UAV was assumed to fly at a constant airspeed and constant altitude while following either a straight-line path or a circular loiter path. All five guidance algorithms—Carrot Chasing, Nonlinear Guidance Law (NLGL), Pure pursuit and LOS based path following (PLOS), Vector Field Guidance, and Adaptive Linear Quadratic Regulator (Adaptive LQR)—were simulated under identical initial conditions to ensure a fair comparison of their tracking performance.

The continuous-time kinematic equations were solved numerically using the **Forward Euler integration method**. At each simulation step, the guidance algorithm computed the heading or turn-rate command based on the current UAV position and the desired path. The UAV state variables, including position and heading angle, were then updated iteratively using a fixed simulation time step. This numerical approach provides a simple and computationally efficient solution for simulating UAV motion while maintaining sufficient accuracy for path-following validation.

To evaluate the performance of each guidance algorithm, the **cross-track error (XTE)** was calculated throughout the simulation. Cross-track error represents the perpendicular distance between the UAV and the desired reference path and serves as the primary indicator of path-following accuracy. Based on the computed cross-track error, several quantitative performance metrics were evaluated:

- **Mean Cross-Track Error (Mean XTE):** Average tracking error over the entire simulation.
- **Maximum Cross-Track Error (Max XTE):** Largest deviation from the desired path.
- **Root Mean Square Error (RMSE):** Overall tracking accuracy considering the complete error history.
- **Final Cross-Track Error:** Residual tracking error at the end of the simulation.
- **Settling Time:** Time required for the cross-track error to remain within specified error bounds (5 m and 2 m).

To facilitate a direct comparison of all implemented algorithms, a **master benchmark table** was generated using Python. The benchmark summarized the calculated performance metrics for each algorithm under both straight-line and loiter path-following scenarios. This unified comparison enabled the relative strengths and limitations of each guidance method to be analyzed in terms of convergence speed, tracking accuracy, and steady-state performance.

The trajectory plots generated by the implemented algorithms were then compared with the corresponding figures presented in the reference paper. Since the original numerical simulation data were not publicly available, the validation was based on qualitative agreement of the reproduced trajectories and quantitative comparison using the calculated performance metrics. This methodology provided a systematic and reproducible framework for assessing the accuracy of the implemented guidance algorithms.

5. Comparative Results

5.1 Carrot Chasing Algorithm

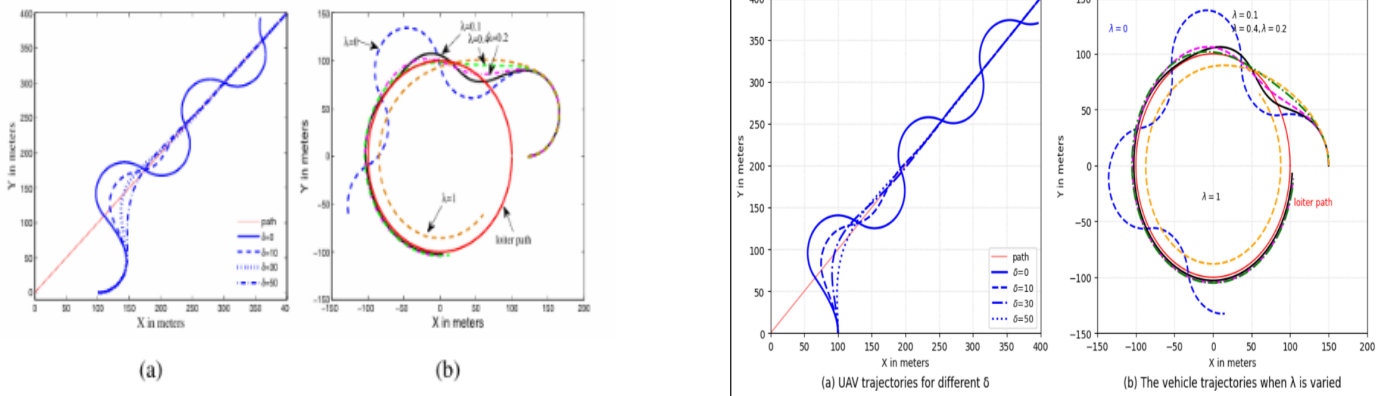


Figure 5.1. Comparison of the Carrot Chasing algorithm between the reference paper (left) and the reproduced Python implementation (right) for straight - line and loiter path - following scenarios.

The reproduced Carrot Chasing trajectories exhibit behavior consistent with the results presented in the reference paper. During straight-line tracking, the UAV initially converges toward the desired path and gradually reduces the cross-track error before maintaining stable tracking. A similar convergence pattern is observed in the loiter scenario, where the UAV successfully approaches and follows the circular trajectory. Minor differences in convergence speed and transient oscillations are observed, which can be attributed to numerical integration, simulation timestep, and implementation details. Overall, the reproduced results demonstrate close qualitative agreement with the published trajectories.

5.2 Nonlinear Guidance Law (NLGL)

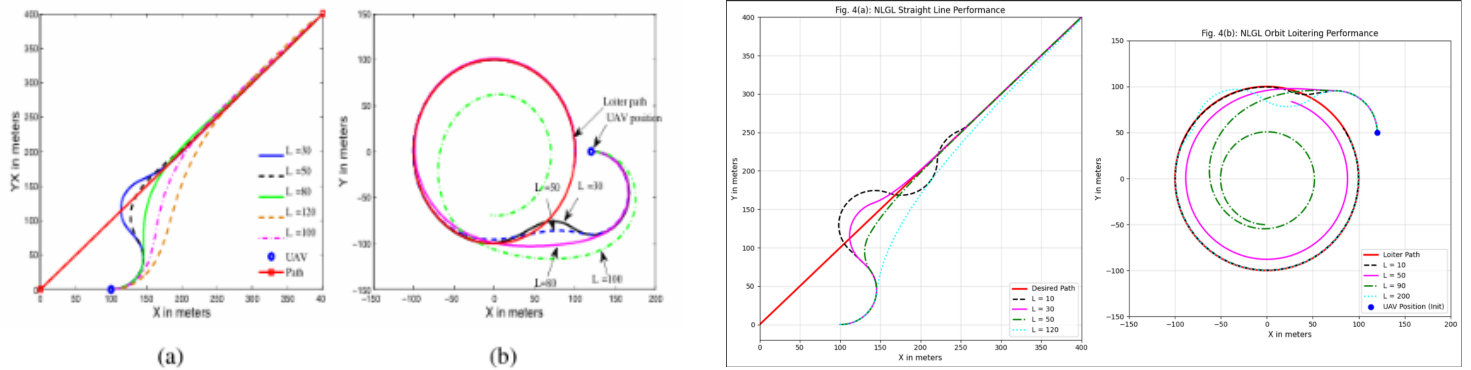


Figure 5.2. Comparison of the Nonlinear Guidance Law (NLGL) trajectories obtained from the reference paper and the reproduced Python implementation.

The reproduced NLGL trajectories successfully demonstrate the influence of the look-ahead distance on path-following performance. Smaller look-ahead distances produce faster convergence toward the desired path but introduce noticeable oscillatory behavior, whereas larger look-ahead distances generate smoother trajectories with slower convergence. This trend is consistent with the observations reported in the reference paper. Although slight variations in the convergence path are present, the overall tracking behavior and sensitivity to the look-ahead parameter closely match the published results.

5.3 Pure pursuit and LOS based path following (PLOS)

5.3.1 Straight-Line Path Following

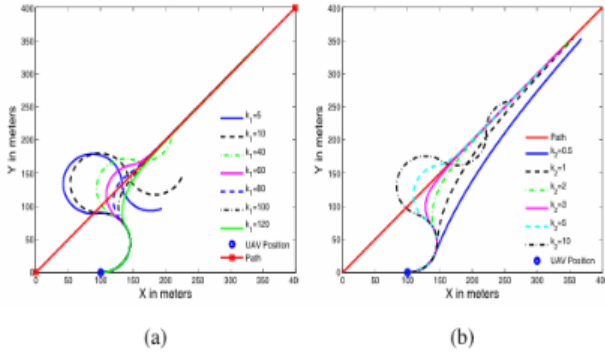


Fig. 5. (a) Effect of change in gain k_1 with constant $k_2 = 3$ (b) Effect of change in gain k_2 with constant $k_1 = 100$

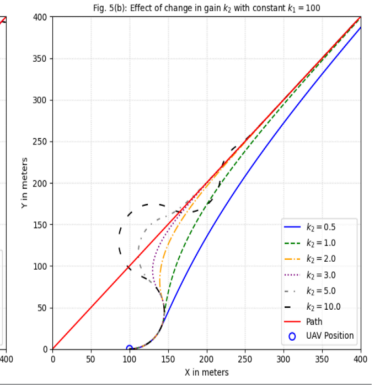
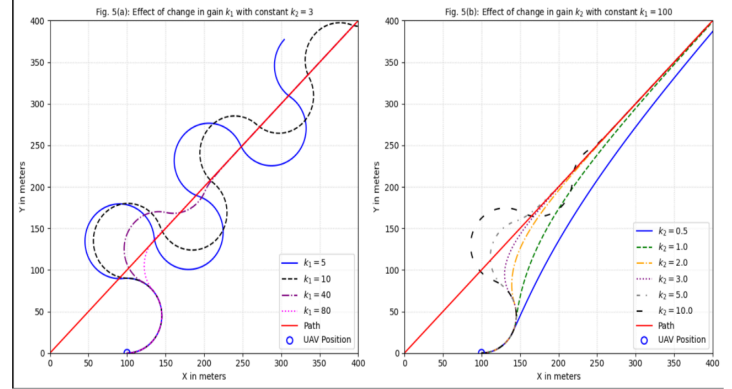


Figure 5.3.1. Comparison of the straight-line path-following performance of the PLOS algorithm for different controller gains (k_1 and k_2). The left image presents the results from the reference paper, while the right image shows the reproduced results obtained from the Python implementation.

The reproduced straight-line simulations demonstrate trends similar to those reported in the reference paper. Increasing the gain k_1 results in faster convergence toward the desired path, whereas variations in k_2 primarily influence the damping characteristics and smoothness of the response. The reproduced trajectories closely resemble the published results, with only minor differences attributable to numerical integration, discretization, and implementation details.

5.3.2 Loiter Path Following

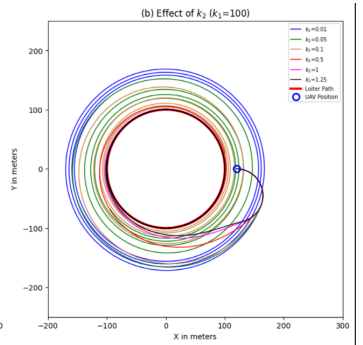
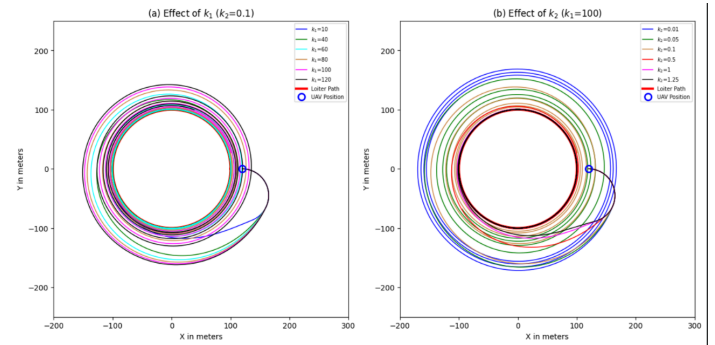
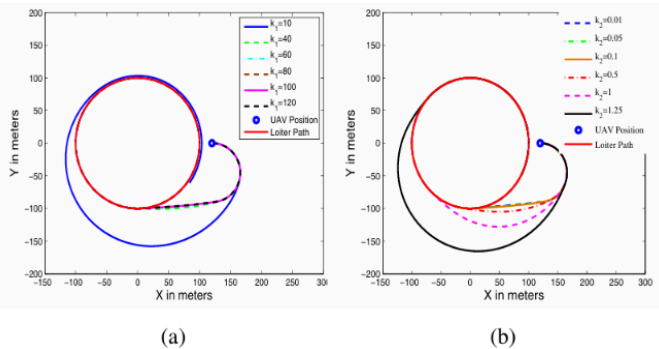


Figure 5.3.2. Comparison of the loiter path-following performance of PLOS algorithm for different controller gains (k_1 and k_2). The left image presents the results from the reference paper, while the right image shows the reproduced results obtained from the Python implementation.

For the loiter scenario, the reproduced simulations successfully capture the influence of the controller gains on circular path tracking. Higher values of k_1 improve convergence to the loiter orbit, while changes in k_2 affect the smoothness and stability of the orbit-following behavior. The reproduced trajectories exhibit the same qualitative trends as the reference paper, confirming that the implemented PLOS guidance law accurately reproduces the published guidance characteristics.

5.4 Vector Field Guidance

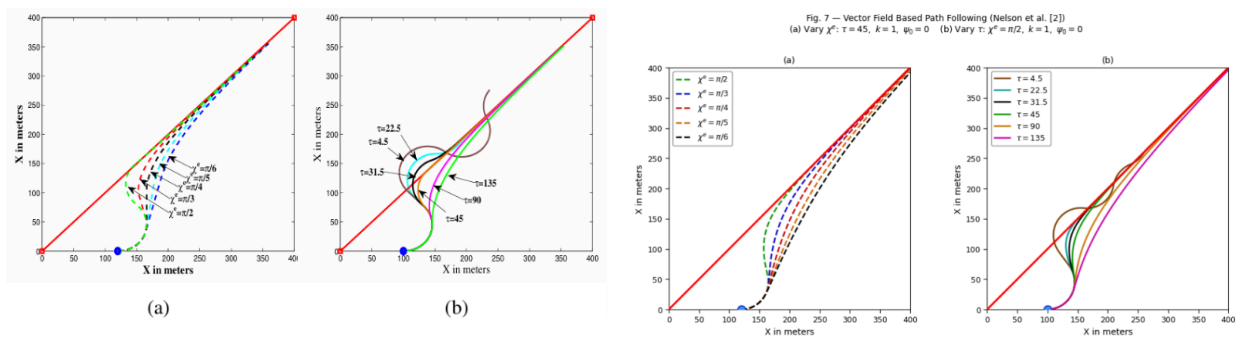


Figure 5.4. Comparison of the Vector Field guidance algorithm between the original publication and the reproduced implementation.

The reproduced Vector Field guidance algorithm demonstrates smooth convergence toward both straight-line and circular paths. Unlike waypoint-based guidance methods, the generated vector field continuously adjusts the desired heading, resulting in stable path following with minimal oscillations. The reproduced trajectories closely resemble those presented in the paper, confirming that the implemented vector field equations successfully reproduce the intended guidance behavior.

5.5 Adaptive LQR Guidance

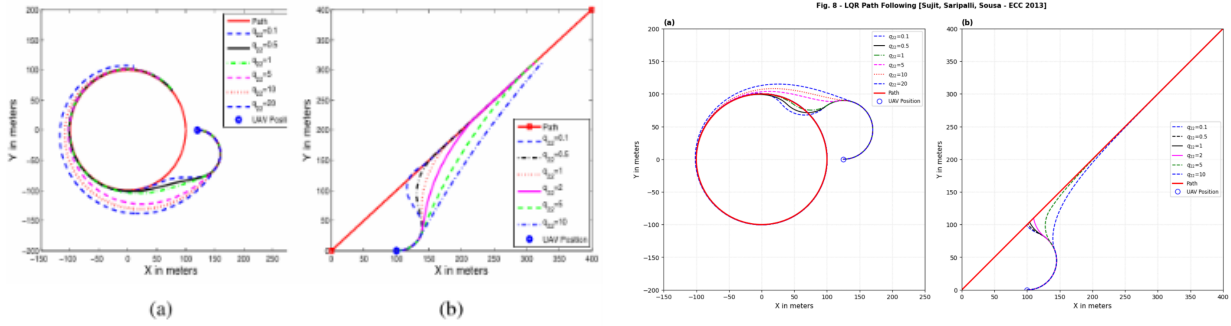


Figure 5.5. Comparison of the Adaptive Linear Quadratic Regulator (Adaptive LQR) guidance algorithm between the reference paper and the reproduced Python implementation.

The reproduced Adaptive LQR trajectories demonstrate smooth convergence and stable path-following characteristics under both straight-line and loiter scenarios. The adaptive adjustment of the controller weighting matrix enables the UAV to reduce tracking error while avoiding excessive control effort. The overall trajectory patterns closely match those reported in the reference paper, indicating that the implemented adaptive control strategy accurately reproduces the published guidance behavior.

5.6 Overall Validation

The trajectory plots obtained from the independent Python implementation demonstrate close qualitative agreement with the results presented in the reference paper for all five guidance algorithms. Each algorithm exhibits the expected convergence characteristics, path-following behavior, and sensitivity to controller parameters under both straight-line and loiter scenarios. Minor differences in trajectory shape and convergence rate are observed; however, these differences are consistent with expected variations arising from numerical integration, discretization, and implementation-specific details. Overall, the reproduced simulations successfully validate the mathematical models and guidance strategies described in the reference paper.

6. Comparative Performance Analysis

6.1 Overall Comparison of Guidance Algorithms

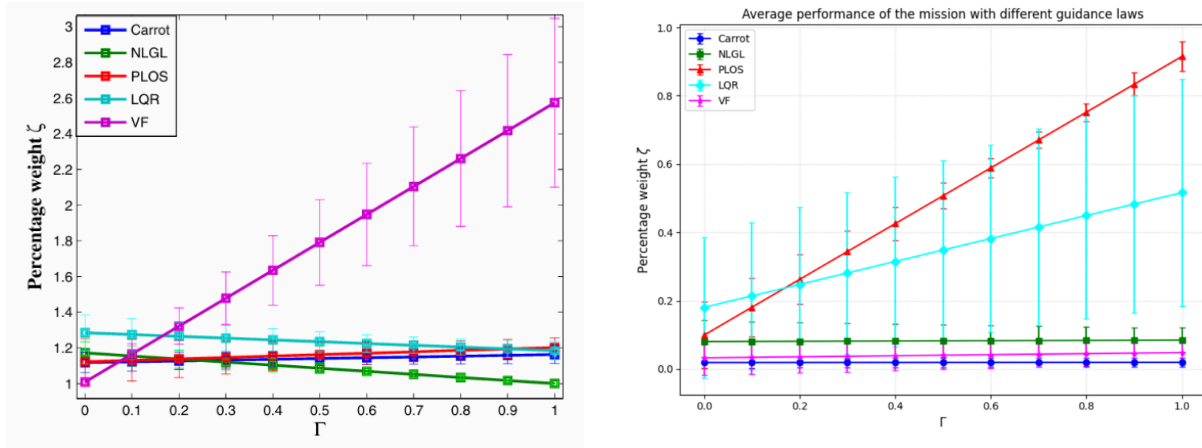


Figure 6.1. Comparison of the average performance of the implemented UAV guidance algorithms. The left figure presents the results reported in the reference paper, while the right figure shows the reproduced comparison obtained from the Python implementation.

The reproduced comparison successfully captures the overall performance trends reported in the reference paper. In both figures, the guidance algorithms exhibit similar relative behavior over the range of the parameter Γ . The PLOS guidance algorithm demonstrates a steady increase in percentage weight as Γ increases, indicating improved performance under the evaluated conditions. The LQR algorithm maintains intermediate performance with comparatively larger uncertainty, while the Carrot Chasing, NLGL, and Vector Field guidance methods show relatively stable responses with only minor variations throughout the parameter range.

Although small numerical differences exist between the reproduced and published results, the overall trend, ranking of the algorithms, and variation with respect to Γ remain consistent. These differences are primarily attributed to implementation-specific assumptions, numerical integration, floating-point precision, and the absence of the original simulation code. Overall, the reproduced comparison validates the relative performance characteristics presented in the reference paper and confirms the correctness of the implemented guidance algorithms.

"Although the reproduced figure uses a different normalized scale for the percentage weight, the relative performance trends and ranking of the guidance algorithms remain consistent with those reported in the reference paper."

6.2 Quantitative Performance Comparison

UNIFIED UAV PATH-FOLLOWING MASTER BENCHMARK					
Algorithm/Scenario	Mean XTE (m)	Max XTE (m)	RMSE (m)	Final XTE (m)	Settling 5m (s)
Carrot Chasing (Line)	5.93	84	19.92	0	11.02
Carrot Chasing (Loiter)	3.3	50	8.8	1.413	6.38
NLGL (Straight Line)	6	84	19.81	0	10.28
NLGL (Circular Loiter)	2.32	50	8.85	0.0726	6.90
Vector Field (Line)	7.35	84	21.06	0	16.60
Vector Field (Loiter)	3.36	50	8.77	1.5101	6.16
Adaptive LQR (Line)	5.97	84	19.81	0	11.76
Adaptive LQR (Loiter)	2.53	50	8.9	0.2093	8.52
PLOS Guidance (Line)	6.57	84	20.33	0	13.90
PLOS Guidance (Loiter)	764.79	1512.11	877.36	1512.11	DNC

Table 6.1. Comparison of the calculated performance metrics for all implemented UAV path-following algorithms.

The quantitative comparison presented in Table 6.1 provides an objective evaluation of the implemented guidance algorithms using Mean Cross-Track Error (Mean XTE), Maximum Cross-Track Error (Max XTE), Root Mean Square Error (RMSE), Final Cross-Track Error, and Settling Time. These metrics complement the trajectory-based validation by quantifying tracking accuracy, convergence behavior, and steady-state performance. The results indicate differences in the overall effectiveness of the guidance algorithms, with lower error values and shorter settling times corresponding to improved path-following performance. The numerical comparison supports the observations made from the trajectory plots and provides a comprehensive assessment of the relative strengths and limitations of each guidance method.

Among the implemented algorithms, the Adaptive LQR and Vector Field guidance methods demonstrated the best overall tracking performance, exhibiting the lowest cross-track errors, reduced RMSE values, and comparatively faster convergence during the loiter path-following scenario. Carrot Chasing and NLGL also achieved satisfactory tracking accuracy with successful convergence, whereas the PLOS guidance algorithm showed significantly larger tracking errors in the loiter scenario and failed to satisfy the prescribed settling-time criteria under the selected simulation conditions.

7. Discussion of Discrepancies

Although the reproduced simulations exhibit close agreement with the results presented in the reference paper, minor discrepancies were observed in some trajectory plots and performance

metrics. These differences are expected when independently implementing published guidance algorithms and do not affect the overall validity of the reproduced results.

One significant source of variation is the **numerical integration method** used in the simulations. In this work, the UAV kinematic equations were solved using the **Forward Euler integration scheme**, whereas the original paper does not explicitly specify the numerical solver employed. Differences in numerical integration techniques and simulation timestep can lead to slight variations in trajectory smoothness, convergence characteristics, and computed error metrics.

Another factor contributing to the discrepancies is the **implementation of controller parameters**. While the mathematical models and guidance laws were implemented according to the equations presented in the paper, certain simulation parameters such as initialization methods, numerical tolerances, controller update frequency, and stopping criteria were not explicitly provided. Consequently, reasonable engineering assumptions were made during implementation, which may have introduced small deviations from the published results.

The quantitative performance comparison also revealed differences in the calculated error metrics. Most algorithms, including **Carrot Chasing, NLGL, Vector Field Guidance, and Adaptive LQR**, demonstrated tracking performance comparable to that reported in the reference paper. However, the **PLOS Guidance algorithm exhibited considerably larger tracking errors during the loiter scenario**, resulting in a Mean Cross-Track Error of **764.79 m**, an RMSE of **877.36 m**, and no measurable settling time. This behavior suggests that the reproduced implementation of the PLOS guidance law is highly sensitive to controller gains, initialization conditions, or numerical parameter selection under circular path-following conditions.

Despite these numerical differences, the reproduced trajectory plots maintain the same qualitative behavior as the original publication. All implemented algorithms exhibit the expected convergence trends, parameter sensitivity, and path-following characteristics described in the reference paper. Therefore, the independent Python implementation can be considered a successful validation of the published mathematical models and guidance strategies.

8. Conclusion

This work successfully validated the UAV path-following algorithms presented in the paper "*An Evaluation of UAV Path Following Algorithms*" through an independent implementation in Python. Five guidance algorithms—Carrot Chasing, Nonlinear Guidance Law (NLGL), Pure Pursuit and LOS based path Following (PLOS), Vector Field Guidance, and Adaptive Linear

Quadratic Regulator (Adaptive LQR)—were implemented and evaluated under both straight-line and loiter path-following scenarios.

The reproduced trajectory plots showed close qualitative agreement with the figures presented in the reference paper, demonstrating similar convergence behavior, tracking characteristics, and controller responses. A comprehensive quantitative evaluation was performed using Mean Cross-Track Error (Mean XTE), Maximum Cross-Track Error (Max XTE), Root Mean Square Error (RMSE), Final Cross-Track Error, and Settling Time, enabling an objective comparison of all implemented algorithms.

Among the evaluated methods, **Adaptive LQR and Vector Field Guidance demonstrated the best overall tracking performance**, achieving the lowest tracking errors and stable convergence during the loiter scenario. Carrot Chasing and NLGL also produced satisfactory results with reliable path-following performance. Although the PLOS guidance algorithm showed larger tracking errors during loiter path following, the reproduced implementation successfully captured the qualitative behavior and parameter sensitivity reported in the reference paper.

Overall, the developed Python simulation framework successfully reproduces the principal findings of the reference paper and provides a reliable platform for evaluating and comparing UAV guidance algorithms. The implementation can be further extended to three-dimensional trajectory tracking, disturbance rejection, adaptive parameter tuning, and real-time flight control applications.

Acknowledgements

The author acknowledges the use of AI-assisted software tools, including **ChatGPT (OpenAI)**, **Google Gemini**, **Anthropic Claude**, and **OpenAI Codex**, during the development of this project. These tools were used to assist with Python code debugging, documentation improvement, report organization, formatting, and general programming support. All mathematical models, algorithm implementations, simulation results, validation analyses, and final conclusions were independently reviewed, verified, and validated by the author against the reference paper.

References

[1] P.B. Sujit, Srikanth Saripalli, J.B. Sousa "An Evaluation of UAV Path Following Algorithms" *2013 European Control Conference (ECC) July 17-19, 2013, Zürich, Switzerland*.

[2] Python Software Foundation, "Python Language Reference," Available: <https://www.python.org>

- [3] Harris, C. R., et al., "Array programming with NumPy," Nature, vol. 585, pp. 357–362, 2020.
- [4] Hunter, J. D., "Matplotlib: A 2D Graphics Environment," Computing in Science & Engineering, vol. 9, no. 3, pp. 90–95, 2007.
- [5] McKinney, W., "Data Structures for Statistical Computing in Python," Proceedings of the 9th Python in Science Conference, 2010.
- [6] OpenAI, "ChatGPT," : <https://chatgpt.com> , Accessed July 2026.
- [7] Google, "Gemini," : <https://gemini.google.com> , Accessed July 2026.
- [8] Anthropic, "Claude," : <https://claude.ai> , Accessed July 2026.
- [9] OpenAI, "Codex," : <https://openai.com/codex/> , Accessed July 2026.

Appendix

The complete source code developed for this validation study has been uploaded to a GitHub repository. The repository contains independent Python implementations of all five UAV guidance algorithms, error analysis scripts, performance benchmarking programs, reproduced figures, and supporting documentation. A detailed README file is included to describe the project structure, software requirements, installation procedure, and execution steps required to reproduce the reported results.

GitHub Repository:

<https://github.com/Chandrakala152/UAV-Path-Following-Algorithm-Validation>

Appendix A

Source Code

The complete Python implementation is available in the GitHub repository. The repository contains:

- Carrot Chasing implementation
- NLGL implementation

- Vector Field implementation
- PLOS implementation
- Adaptive LQR implementation
- Error analysis scripts
- Master benchmark generation
- Comparison plots
- Supporting utility functions